CBflow v2.0.0 – Physical Design Automation Framework

Feature Document

Classification: Confidential

Version: 2.0.0

Table of Contents

---

1. Executive Summary

CBflow is a comprehensive, end-to-end ASIC Physical Design automation framework designed to unify the complete PD flow – from RTL input validation through synthesis, place-and-route, static timing analysis, physical verification, and tapeout release – under a single, vendor-agnostic platform.

Modern semiconductor design teams face a persistent challenge: fragmented, project-specific scripting environments that are difficult to maintain, impossible to standardize, and inherently brittle when confronted with tool upgrades, technology node migrations, or team scaling. Individual engineers accumulate personal script libraries, flow knowledge resides in undocumented tribal expertise, and cross-project reuse remains aspirational rather than practical.

CBflow addresses this challenge through architectural rigor rather than incremental scripting improvement. The framework introduces a declarative configuration model where every aspect of the design flow – stage definitions, dependencies, tool bindings, resource allocations, and quality checkpoints – is expressed through structured TCL configuration arrays. This single source of truth eliminates the environment variable proliferation and ad-hoc override mechanisms that have historically plagued PD flow management.

The core differentiator of CBflow v2.0.0 is the custom **RACE engine** (Run Automation and Control Engine) – a Python-native Directed Acyclic Graph (DAG) executor that replaces traditional Makefile-based flow control. RACE provides persistent state tracking via an embedded SQLite database, intelligent re-execution through automatic file change detection, and parallel dispatch capability for independent flow stages. The engine operates with zero external dependencies beyond the Python standard library, ensuring deployment simplicity across any compute infrastructure.

CBflow v2.0.0 supports twelve production-grade design flows spanning the full PD methodology: SYNTH, PNR, SYNTH_PNR, FP, FCFP, STA, PV, LEC, CLP, ECO, EMIR, and POPT. Each flow is defined declaratively, supports multiple EDA vendor backends, and integrates natively with the RACE engine for execution, monitoring, and release management.

---

2. Architecture Overview

2.1 Layered Architecture

CBflow is organized into four distinct architectural layers, each with clearly defined responsibilities and well-specified interfaces:

```
+-------------------------------------------------------------
+
|                         PRESENTATION
LAYER                               |
|  FlowTracer GUI  |  Dashboard  |  CLI Interface  |  Email/
Reports    |
+-------------------------------------------------------------
+
|                          ENGINE
LAYER                                    |
|  RACE Engine (DAG Executor)  |  StatusDB (SQLite)  |  Job
Dispatcher |
+-------------------------------------------------------------
+
|                         EXECUTION
LAYER                                  |
|  TCL Handlers (per vendor/tool/version)  |  Subnode
Handlers         |
+-------------------------------------------------------------
+
```

```
|                          CONFIGURATION
LAYER                              |
|  flow_config  |  node_config  |  project_config  |
tech_config      |
|  user_config  |  override_config  |  mmmc_config  |
tool_launch     |
+-------------------------------------------------------------
+
```

Configuration Layer. The foundation of the framework. All flow behavior is defined through structured TCL configuration files organized by scope and precedence. This layer establishes the single source of truth for every configurable parameter in the system.

Execution Layer. TCL-based tool handlers that translate abstract stage definitions into concrete EDA tool invocations. Handlers are organized by vendor, tool name, and tool version, enabling precise version control and seamless tool migration.

Engine Layer. The RACE engine builds and executes a DAG derived from configuration-defined stage dependencies. The embedded SQLite database provides persistent state tracking, crash recovery, and historical execution data.

Presentation Layer. Browser-based visualization (FlowTracer GUI), command-line status reporting, automated email notifications, and PowerPoint report generation provide multiple interaction modalities for different user roles and contexts.

2.2 Configuration Hierarchy

CBflow employs an eight-level configuration cascade that provides maximum flexibility while maintaining predictability. Each level has a well-defined scope, and later levels override earlier ones:

Priority	Configuration Level	Scope	Description
1 (base)	flow_config.tcl	Framework-wide	Flow version, global settings, enabled features
2	node_config.tcl	Per-flow	Stage definitions, dependencies, subnodes, tool bindings
3	project_config.tcl	Per-project	Technology node, standard cells, project paths
4	tech_config.tcl	Per-technology	Process-specific parameters, layer maps, DRC rules
5	mmmc_config.tcl	Per-project	MMMC scenario definitions, corner/mode associations

Priority	Configuration Level	Scope	Description
6	user_config.tcl	Per-run	User inputs, file paths, design-specific overrides
7	override_config.tcl	Per-stage	Stage-level parameter overrides (memory, CPU, timeout)
8 (top)	runtime_flow_config.tcl	Per-run	Dynamic nodes, branches, runtime modifications

This hierarchy ensures that global framework defaults propagate cleanly to individual runs while permitting fine-grained customization at every level. Importantly, CBflow does not rely on environment variable overrides for configuration – all parameters are resolved from the TCL configuration chain, providing auditability and reproducibility.

2.3 Multi-Vendor Abstraction

CBflow decouples flow definition from tool implementation through a handler abstraction layer. The same stage structure (e.g., synthesis, place, CTS, route) applies regardless of whether the backend tool is Synopsys Fusion Compiler, Cadence Innovus, or any other supported EDA tool. Tool-specific behavior is encapsulated in handler scripts organized by a strict directory convention:

```
cmds/<FLOW_TYPE>/<vendor>/<tool>/<version>/
<node_type>_subnode_handler.tcl
```

This architecture enables tool switching through a single configuration change, with no modifications to the flow definition, dependency graph, or monitoring infrastructure.

2.4 Directory Structure Conventions

CBflow enforces a deterministic directory structure for every run, ensuring that outputs, logs, reports, and intermediate data are always located in predictable paths:

```
<run_directory>/
    setup/              Configuration files (user_config,
override_config, runtime_flow_config)
    work/               Per-stage working directories (work/
<stage>/<subnode>/)
    results/            Final output data (GDS, netlists,
DEF, SPEF)
    reports/            Analysis reports (timing, power,
DRC, LVS)
    .stamps/            Completion stamps (backward
compatibility with external tools)
```

```
.race_<uid>.db      RACE engine status database
.run.cbflow.env     Run environment snapshot
```

---

3. RACE Engine – Core Innovation

3.1 What is RACE

The **Run Automation and Control Engine** (RACE) is a Python-native DAG executor purpose-built for ASIC Physical Design flow orchestration. RACE replaces the traditional approach of using GNU Make or shell script chains for flow control – an approach that, while ubiquitous in the industry, suffers from fundamental limitations in state management, error recovery, parallel execution, and operational visibility.

RACE was designed from the ground up with the specific requirements of PD flow management:

- **Persistent state tracking.** Every job execution is recorded in an embedded SQLite database with full metadata: status, timing, exit codes, host information, LSF job identifiers, and error messages. This provides complete audit trails and enables intelligent re-execution decisions.
- **Crash recovery.** The engine resumes from the last completed state after any interruption – whether due to process termination, system failure, or manual halt. No re-execution of previously completed stages is necessary.
- **Intelligent re-execution.** RACE monitors input file modifications and automatically determines which downstream stages require re-execution, eliminating both unnecessary reruns and missed dependencies.
- **Zero external dependencies.** RACE is implemented entirely in Python 3.8+ using only the standard library (os, sqlite3, subprocess, signal, pathlib). No pip packages, no virtual environments, no package management overhead. The engine deploys by file copy alone.

3.2 DAG Architecture

RACE constructs a two-level Directed Acyclic Graph from the declarative stage and dependency definitions in each flow's node configuration file.

Stage Level (Macro Steps)

At the top level, the DAG consists of **stages** – the major steps of a design flow. For example, the SYNTH_PNR flow defines thirteen stages:

```
rtl1 -> sdc1 -> upf1 -> init_design1 -> synthesis1 -> place1
-> cts1 ->
```

```
cts_opt1 -> route1 -> pro1 -> signoff1 -> export_data1 ->
release_data1
```

Stage dependencies are declared explicitly in configuration. Input stages (rtl1, sdc1, upf1) have no dependencies and can execute in parallel. The init_design1 stage depends on all three inputs. Subsequent stages form a linear pipeline through the core PD flow.

Subnode Level (Micro Steps)

Each execution stage is further decomposed into **subnodes** – fine-grained steps that map to individual tool handler invocations. The standard subnode pattern is:

- **setup** – Environment preparation, file staging, configuration validation
- **run** – Primary tool execution (synthesis, placement, routing, etc.)
- **validate** – Output verification, QoR metric extraction, checklist evaluation
- **finish** – Result packaging, report generation, stamp creation

Subnode dependencies are independently defined, enabling intra-stage parallelism where the dependency structure permits it.

Leaf Nodes

Input stages (RTL, SDC, UPF, netlist, DEF, GDS, library, SPEF) are classified as **leaf nodes**. These stages have no subnodes and execute directly at the stage level – performing input validation and file registration without the overhead of the setup/run/validate/finish subnode pipeline. This design ensures that input processing is lightweight and fast.

Dynamic Subnodes

Certain stages support **dynamic subnode resolution** at runtime. The most significant application is in the STA flow's timing analysis stage, where MMMC (Multi-Mode Multi-Corner) scenarios are resolved from the user's configuration at initialization time. If a user defines four signoff scenarios (e.g., func_wc_rcmax, func_bc_rcmin, func_wc_rcmin, func_bc_rcmax), RACE automatically generates subnodes:

```
setup -> func_wc_rcmax -> func_bc_rcmin -> func_wc_rcmin ->
func_bc_rcmax -> validate -> finish
```

The scenario subnodes depend only on setup and are independent of each other, enabling fully parallel timing analysis across all MMMC corners and modes.

Topological Execution Guarantee

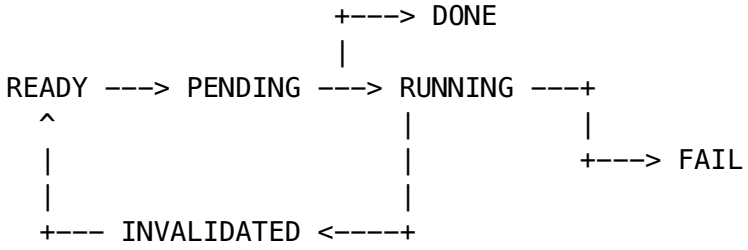
RACE performs topological sorting of the complete job DAG before execution begins. This sorting, combined with the explicit dependency declarations in configuration, provides a mathematical guarantee that no job

executes before all of its dependencies have completed successfully. Circular dependencies are detected at initialization time and reported as configuration errors. Deadlock is structurally impossible.

3.3 Execution Model

Status Lifecycle

Every job in the RACE DAG transitions through a well-defined set of states:




```
Special states (set by operator):
  BYPASSED           -- Stage skipped (not needed for this
run)
  FORCE_VALIDATED    -- Stage marked complete (executed
externally)
```

READY. Initial state. The job exists in the DAG but has not been scheduled for execution.

PENDING. The job has been scheduled for the current execution run. All pending jobs are recorded to the database before execution begins, providing a complete manifest of planned work.

RUNNING. The job's handler is actively executing. Start time, process ID, and hostname are recorded.

DONE. The handler exited with code zero. End time, runtime duration, and host information are recorded.

FAIL. The handler exited with a non-zero code. The error message, exit code, and full execution metadata are preserved for diagnosis.

INVALIDATED. The job's output is no longer valid – either because an upstream dependency was modified, input files changed, or an operator explicitly requested re-execution. Invalidated jobs return to the execution pipeline.

BYPASSED. The job was explicitly skipped by an operator. RACE treats bypassed jobs as completed for dependency resolution, allowing downstream stages to proceed.

FORCE_VALIDATED. The job was marked as completed by an operator without actual execution. This state accommodates stages that were run outside the CBflow framework (manual tool invocation, external scripts, imported data from another run) and need to be recognized as complete within the DAG.

Real-Time State Persistence

All status transitions are persisted to the SQLite database in real time. There is no batch update window and no risk of status loss. If the engine process terminates unexpectedly at any point, the database accurately reflects the last known state of every job. Upon restart, the engine reads the database, identifies all jobs in completed states, and resumes execution from the next pending job.

Error Handling

When a job fails, RACE immediately halts execution of downstream jobs that depend on the failed stage. All remaining PENDING jobs that cannot proceed are reset to READY in the database, preserving a clean state for diagnosis and re-execution. The engine reports the specific failure, including exit code and error message, enabling rapid root-cause identification.

3.4 Parallelism

RACE is architected for parallel execution at multiple granularity levels:

Intra-Stage Parallelism (Subnode Level)

When a stage contains multiple subnodes with independent dependency structures, RACE can dispatch them concurrently. This is the primary parallelism mechanism in CBflow v2.0.0 and provides significant runtime reduction for several key flows:

STA Timing Analysis. The timing1 stage uses dynamic subnodes resolved from MMMC scenario definitions. Each scenario (corner/mode combination) is an independent analysis that can run simultaneously. A design with eight signoff scenarios achieves up to 8x speedup on this stage when sufficient compute resources are available.

Physical Verification. The PV flow's verification stages – DRC, LVS, ERC, PERC, and XOR – all depend on the fill1 stage but are independent of each other. After metal fill completes, all five verification checks dispatch in parallel, with the merge_data1 stage waiting for all five to complete before consolidating results.

Input Validation. Input leaf stages (RTL, SDC, UPF, netlist, DEF, GDS, library, SPEF) have no inter-dependencies within a flow. All input stages execute in parallel at the start of every flow, minimizing input processing latency.

Inter-Stage Parallelism (Future)

The RACE architecture supports inter-stage parallelism through LSF (Load Sharing Facility) job dispatch. Stages that reside on independent branches of the DAG can execute on separate compute hosts simultaneously. This capability is under active development for production deployment.

3.5 Smart Operations

RACE provides four operator-initiated smart operations that go beyond simple run/stop control:

Retrace

Retrace invalidates a specified stage and all of its downstream dependents, then re-executes the affected portion of the DAG. This is the correct response to any upstream change – whether a modified SDC constraint file, an updated floorplan, or a revised technology library.

Retrace operates on DAG structure, not on file timestamps. When an operator requests retrace from a specific stage, RACE traverses the dependency graph forward from that point and invalidates every reachable job. This ensures complete and correct propagation regardless of file system state.

Bypass

Bypass marks specified stages as completed without executing them. This operation is essential for iterative development workflows where certain stages are known to be unnecessary for the current experiment. For example, an engineer optimizing placement may bypass CTS and routing stages to accelerate iteration on placement parameters.

Bypassed stages satisfy dependency requirements for downstream stages, enabling the flow to continue past skipped stages without breaking the DAG contract.

Force

Force re-executes specific stages regardless of their current status. Unlike retrace, force does not propagate invalidation to downstream stages. This operation is appropriate when a stage needs re-execution (perhaps with modified parameters) but its downstream results are still considered valid.

Force Validate

Force validate marks stages as completed, recording them with FORCE_VALIDATED status. This accommodates a common production scenario: a stage was executed outside the CBflow framework (direct tool invocation, manual ECO, imported data) and needs to be recognized within the flow’s state model so that downstream stages can proceed.

3.6 File Change Detection

On every initialization, the RACE engine performs automatic file change detection. For each stage that was previously completed (status DONE in the database), the engine compares the stage's completion timestamp against the modification times of files in the stage's working directory.

If any file has been modified after the stage completed, RACE treats this as an implicit input change and automatically triggers a retrace from that stage. This mechanism catches scenarios such as:

- An engineer modifying an SDC constraint file between runs
- A library update that changes timing models
- A floorplan revision that affects all downstream stages

The detection is conservative: any modification triggers retrace, ensuring that stale results are never propagated downstream.

3.7 Persistent State Database

Every RACE-managed run maintains an SQLite database as its authoritative state store. The database is identified by a deterministic 6-character hexadecimal UID derived from the run directory's absolute path, ensuring that the same run always maps to the same database file.

Database Schema

jobs table. The primary execution record. Every job state transition generates a row, creating a complete historical record. Fields include: job name, stage, subnode, job type, status, command, start time, end time, runtime (seconds), exit code, LSF job ID, LSF queue, resource tier, process ID, hostname, CPU utilization, peak memory (MB), log file path, error message, retry count, and creation timestamp.

run_info table. Key-value metadata for the run: flow type, run directory, project name, user, initialization time, execution target, start time, completion time, and result status.

stage_metrics table. Quality-of-results metrics captured per stage: timing slack, area, power, cell count, wire length, congestion metrics, and any other QoR data extracted by validate subnodes.

job_order table. The canonical DAG ordering, stored as a sequence number per job. This table preserves the topological sort order and enables the GUI and reporting tools to present stages in the correct flow sequence.

Historical Preservation

The database preserves all execution history. Status queries resolve the latest state via MAX(id) per job name, while the full history remains available for trend analysis, runtime estimation, and debugging. This append-only design ensures that no execution data is ever lost, even through multiple retrace and re-execution cycles.

Structured Database Path

When a project-level database path is configured, RACE organizes databases in a structured hierarchy:

```
<db_base_path>/<project_name>/<domain>/<flow_type>/
<user>_<run_name>_<uid>.db
```

This convention enables centralized database management, cross-run analytics, and project-wide QoR tracking without requiring shared file system access to individual run directories.

---

4. Supported Design Flows

CBflow v2.0.0 provides twelve production-grade design flows spanning the complete ASIC Physical Design methodology. Each flow is defined declaratively through a dedicated node configuration file that specifies stages, dependencies, subnodes, tool bindings, and runtime parameters.

4.1 SYNTH – Logic Synthesis

Standalone logic synthesis flow for RTL-to-gate compilation.

Property	Value
Stages	7 (rtl1, sdc1, upf1, init_design1, synthesis1, export_data1, release_data1)
Parallel Capability	3 input stages execute in parallel
Vendor Support	Synopsys Fusion Compiler, Cadence Genus
Key Features	Merge-capable (entry/handoff stages for composite flows), UPF power-aware synthesis

4.2 PNR – Place and Route

Standalone place-and-route flow from gate-level netlist through signoff.

Property	Value
Stages	13 (netlist1, sdc1, def1, upf1, init_design1, place1, cts1, cts_opt1, route1, pro1, signoff1, export_data1, release_data1)

Property	Value
Parallel Capability	4 input stages execute in parallel
Vendor Support	Synopsys Fusion Compiler, Cadence Innovus
Key Features	Merge-capable, full pipeline from placement through signoff, FC-RM stage alignment

4.3 SYNTH_PNR – Unified Synthesis + Place and Route

Integrated synthesis and place-and-route flow leveraging Fusion Compiler's unified optimization capability.

Property	Value
Stages	13 (rtl1, sdc1, upf1, init_design1, synthesis1, place1, cts1, cts_opt1, route1, pro1, signoff1, export_data1, release_data1)
Parallel Capability	3 input stages execute in parallel; MMMC-aware with multi-corner compile
Vendor Support	Synopsys Fusion Compiler (primary unified flow)
Key Features	FC-RM Y-2026.03 aligned stage names, endpoint optimization, fusion-advanced extraction, QoR strategy initialization, complete output set (GDS, netlist, DEF, SPEF)

4.4 FP – Floorplanning

Block-level floorplanning flow covering design import through pin placement.

Property	Value
Stages	12 (netlist1, sdc1, def1, upf1, library1, init_design1, import_design1, floorplan1, powerplan1, place_pins1, export_data1, release_data1)
Parallel Capability	5 input stages execute in parallel
Vendor Support	Synopsys Fusion Compiler, Cadence Innovus
Key Features	Merge-capable (entry/handoff for composite flows), dedicated power planning and pin placement stages

4.5 FCFP – Full-Chip Floorplanning (Hierarchical Design Planning)

Hierarchical design planning flow aligned with the FC-RM dp_hier methodology.

Property	Value
Stages	17 (netlist1, sdc1, def1, upf1, library1, init_design1, commit_blocks1, init_compile1, create_floorplan1, shaping1, placement1, create_power1, place_pins1, top_compile1, timing_budget1, export_data1, release_data1)
Parallel Capability	5 input stages execute in parallel
Vendor Support	Synopsys Fusion Compiler (FC-RM DP Hier)
Key Features	Full FC-RM dp_hier alignment, block commitment, shaping, top-level compile, timing budgeting

4.6 STA – Static Timing Analysis

Multi-Mode Multi-Corner (MMMC) static timing analysis with dynamic per-scenario parallelism.

Property	Value
Stages	8 (netlist1, sdc1, spef1, library1, extraction1, timing1, reporting1, release_data1)
Parallel Capability	4 input stages in parallel; timing1 stage runs all MMMC scenarios in parallel (dynamic subnodes)
Vendor Support	Synopsys PrimeTime, Cadence Tempus
Key Features	Dynamic scenario resolution from user config, split setup/hold analysis, AOCV/POCV support, PBA (path-based analysis) exhaustive mode, SI-aware analysis, cross-corner worst-case aggregation

4.7 PV – Physical Verification

Comprehensive physical verification flow with parallel check execution.

Property	Value
Stages	11 (netlist1, def1, gds1, fill1, drc1, lvs1, perc1, erc1, xor1, merge_data1, release_data1)
Parallel Capability	3 input stages in parallel; 5 verification checks (DRC, LVS, PERC, ERC, XOR) execute in parallel after fill
Vendor Support	Synopsys ICV, Mentor/Siemens Calibre, Cadence PVS
Key Features	ICV-RM V-2023.12 alignment, FEOL/BEOL metal fill, foundry runset integration, per-check CPU allocation, comprehensive rule deck support

4.8 LEC – Logic Equivalence Checking

Formal verification flow for golden-vs-revised netlist comparison.

Property	Value
Stages	6 (netlist_golden1, netlist_revised1, constraints1, setup1, compare1, analyze1)
Parallel Capability	3 input stages execute in parallel
Vendor Support	Synopsys Formality
Key Features	Merge-capable, separate golden and revised netlist inputs, constraint-aware comparison

4.9 CLP – Clock-Level Power Analysis

Low-power verification flow for UPF/CPF power intent validation.

Property	Value
Stages	5 (netlist1, upf1, power_spec1, clp1, release_data1)
Parallel Capability	3 input stages execute in parallel
Vendor Support	Synopsys VC-LP, Cadence Conformal LP
Key Features	Merge-capable, UPF power specification validation, power domain analysis

4.10 ECO – Engineering Change Order

Late-stage design modification flow for functional and timing ECOs.

Property	Value
Stages	6 (netlist1, def1, sdc1, library1, eco1, export_db1)
Parallel Capability	4 input stages execute in parallel
Vendor Support	Synopsys Fusion Compiler, Synopsys ICC2
Key Features	Merge-capable, supports both functional and timing ECO methodologies, database export for downstream integration

4.11 EMIR – Electromigration and IR Drop Analysis

Power integrity analysis flow covering static/dynamic IR drop and electromigration.

Property	Value
Stages	7 (netlist1, def1, spef1, library1, power_analysis1, ir_drop1, thermal_analysis1)

Property	Value
Parallel Capability	4 input stages execute in parallel
Vendor Support	Synopsys RedHawk, Cadence Voltus
Key Features	Three-stage analysis pipeline (power, IR drop, thermal), full power grid analysis

4.12 POPT – Post-Route Optimization

Post-route timing optimization flow for PrimeTime-driven ECO.

Property	Value
Stages	7 (netlist1, sdc1, upf1, merge_timing1, power_opt1, post_merge1, release_data1)
Parallel Capability	3 input stages execute in parallel
Vendor Support	Synopsys PrimeTime
Key Features	Timing merge, power optimization, post-merge signoff integration

5. FlowTracer GUI

5.1 Overview

CBflow includes a browser-based flow visualization and control interface modeled after the industry-standard FlowTracer paradigm. The GUI requires zero client-side installation – it is served by a Python standard library HTTP server and accessed from any modern web browser on any device with network access to the compute environment.

The GUI is implemented entirely in HTML5, CSS3, and JavaScript with no external library dependencies (no React, no Angular, no jQuery). This design decision ensures long-term maintainability, eliminates version compatibility concerns, and allows deployment in restricted network environments where package repositories may be unavailable.

5.2 Three-Panel Layout

The FlowTracer GUI employs a three-panel layout optimized for PD flow monitoring:

Left Panel – Hierarchy Tree. A collapsible tree view displaying all flow stages with their constituent subnodes. Each stage can be expanded to reveal its subnodes (setup, run, validate, finish, or dynamic MMMC scenarios). Stages display color-coded status indicators and runtime information. Click-to-select and keyboard navigation are supported for efficient inspection.

Center Panel – DAG Canvas. An interactive SVG-based dependency graph rendered from the RACE engine's DAG structure. Nodes are color-coded by execution status:

Color	Status	Meaning
Purple	Idle / Ready	Not yet scheduled for execution
Blue	Queued / Pending	Scheduled, awaiting dependency completion
Yellow	Running	Currently executing
Green	Done	Successfully completed
Red	Failed	Execution failed
Orange	Bypassed	Skipped by operator
Teal	Force Validated	Marked complete by operator

Directed edges between nodes represent dependency relationships. The canvas supports zoom, pan, and drag-to-reposition for complex flow layouts. Parallel branches (such as PV verification checks or STA MMMC scenarios) are rendered as fan-out/fan-in patterns that visually communicate the parallelism structure.

Right Panel – Properties. A context-sensitive detail panel that displays comprehensive information for the selected node: execution status, start and end times, runtime duration, exit code, hostname, LSF job ID, resource tier, command string, error message (if failed), and a live log tail showing the last 50 lines of the stage's execution log.

5.3 Real-Time Monitoring

The GUI implements automatic polling with a 5-second refresh interval. Status updates propagate to all three panels simultaneously: the tree updates status indicators, the DAG canvas recolors nodes, and the properties panel refreshes runtime counters. This enables passive monitoring of long-running flows without manual refresh actions.

A status bar at the top of the interface displays aggregate progress: total jobs, completed count, running count, failed count, and overall completion percentage.

5.4 Interactive Node Operations

The GUI supports direct interaction with flow nodes:

- **Single click** selects a node and populates the properties panel with detailed execution metadata.
- **Double click** on a stage node drills down into its subnode view, displaying the internal setup/run/validate/finish pipeline (or dynamic MMMC scenarios) with their individual statuses and runtimes.
- **Context actions** (accessible through the toolbar and dropdown menus) enable operators to trigger retrace, bypass, force, and force-validate operations directly from the GUI without command-line access.

5.5 Inline Configuration Editing

The GUI provides read and edit access to per-stage configuration parameters. For each stage, operators can inspect and modify:

- LSF queue assignment (resource tier mapping)
- Memory allocation
- CPU core count
- Runtime timeout threshold
- Tool version selection

Configuration changes are written to the appropriate override configuration file, ensuring they persist across GUI sessions and engine re-initializations.

5.6 Dynamic Node Management

The FlowTracer GUI supports runtime modification of the flow DAG:

- **Add Node.** Insert a new stage at any point in the flow by specifying its type, dependency, and subnode structure. The node is recorded in the runtime flow configuration and integrated into the RACE engine's DAG.
- **Create Branch.** Fork a parallel execution path from any existing stage. Branches enable experimental flows (e.g., testing an alternative optimization strategy) without disrupting the primary flow.

5.7 Subnode Visualization

Drilling into a stage reveals its subnode structure as a secondary DAG view. For standard stages, this shows the linear setup-run-validate-finish pipeline. For stages with parallel subnodes (such as STA timing analysis with MMMC scenarios), the visualization displays the fan-out pattern: setup fans out to N parallel scenario subnodes, which converge at validate and finish. This visual representation directly communicates the parallelism available in each stage.

5.8 Multiple Views

The GUI offers three complementary views:

- **Dashboard Overview.** High-level summary cards showing project count, release count, active runs, stage performance statistics, and recent run history.
- **DAG View.** The full interactive dependency graph as described above.
- **Grid View.** A sortable, filterable tabular view of all jobs with columns for name, stage, subnode, type, status, runtime, exit code, hostname, LSF job ID, and error message. This view is optimized for bulk status inspection and failed-job identification.

---

6. Flow Customization

6.1 Runtime Node Addition

CBflow supports adding custom stages to a flow at the run level without modifying base configuration files. Custom nodes are defined in the runtime flow configuration and are merged into the RACE engine's DAG at initialization time.

Custom nodes specify: - **Node name** (unique identifier within the flow) - **Node type** (determines which handler is invoked) - **Dependency** (the existing stage after which the custom node executes) - **Subnodes** (defaults to the standard setup/run/validate/finish pattern)

This capability enables per-run experimentation – adding an extra optimization pass, inserting a custom analysis step, or extending the flow with project-specific stages – without creating flow forks or modifying shared configurations.

6.2 Branch Creation

Branches fork parallel execution paths from any point in the flow. A branch creates a new sequence of stages that executes independently of the main flow, sharing the same input data up to the fork point but diverging from that point forward.

Use cases for branching include: - A/B comparison of optimization strategies (e.g., different placement densities) - Running signoff checks on an intermediate state while the main flow continues - Testing alternative tool versions on the same design data

Branches are tracked in the runtime flow configuration with a branch key that enables clear identification in the GUI and reporting tools.

6.3 Override Hierarchy

The eight-level configuration cascade (described in Section 2.2) provides structured customization at every scope. Of particular importance for run-level customization:

- **Per-stage override files** (override_config..tcl) allow individual stages to use different memory allocations, CPU counts, timeouts, or tool parameters without affecting other stages.
- **User configuration** (user_config.tcl) captures all design-specific inputs and parameters that vary between runs.
- **Runtime flow configuration** (runtime_flow_config.tcl) records dynamic modifications including custom nodes and branches.

6.4 Dynamic MMMC Subnodes

For MMMC-aware flows (particularly STA), subnodes are not statically defined in the node configuration. Instead, they are resolved at runtime from the user's MMMC scenario definitions. The RACE engine's DagBuilder reads the user configuration, identifies declared scenarios, and automatically generates the appropriate subnode structure with correct dependency wiring.

This means that adding a new signoff scenario requires only a user configuration change – no flow definition modification, no handler changes, and no DAG restructuring. The engine adapts automatically.

6.5 Per-Stage Configuration Editing

Every configurable parameter – LSF resource allocation, execution timeout, tool version, optimization settings – can be adjusted on a per-stage basis through the override configuration mechanism. The GUI provides inline editing for the most commonly adjusted parameters, while the full configuration hierarchy remains available for advanced customization.

---

7. Input Handshaking and Release Management

7.1 Release Path Convention

CBflow enforces a structured release path convention that provides traceability from initial design exploration through tapeout:

```
<release_base_path>/<phase>/<block_name>/<release_tag>/
```

This convention ensures that every release artifact is uniquely identified by its phase milestone, design block, and version tag.

7.2 Phase Milestones

The CBflow methodology defines four phase milestones that correspond to increasing levels of design maturity:

Phase	Name	Description
P0	Exploration	Initial floorplanning, early synthesis experiments, feasibility studies
P1	Optimization	Active timing closure, placement and routing optimization
P2	Signoff	Full signoff analysis (STA, PV, EMIR), design freeze
P3	Tapeout	Final release, GDS handoff, tapeout-ready deliverables

7.3 Stage Exits

Within each phase, flows define intermediate checkpoints known as stage exits:

- **FP_EXIT** – Floorplan complete, power plan verified, pin placement finalized
- **PLACE_EXIT** – Placement optimized, congestion resolved, timing on track
- **CTS_EXIT** – Clock tree synthesized, skew targets met, hold timing clean
- **PRO_EXIT** – Post-route optimization complete, timing closed
- **BTO** – Block Tapeout – all signoff checks clean for this block
- **MTO** – Macro Tapeout – hierarchical integration complete

7.4 Mandatory File Validation

Each release phase enforces mandatory file requirements. The node configuration defines critical files and mandatory outputs per stage. At release time, CBflow validates that all required artifacts exist and are non-empty before permitting the release to proceed.

For example, the SYNTH_PNR export_data1 stage requires: final GDS layout, final gate-level netlist, final DEF, final SPEF parasitics, and final timing report.

7.5 Cross-Flow Data Handshaking

Flows consume inputs that are outputs of other flows. CBflow manages this handshaking through release tags. A PV flow references a specific SYNTH_PNR release tag for its GDS and netlist inputs. An STA flow references release tags for netlist, SDC, and SPEF. This tag-based referencing ensures reproducibility – re-running an STA flow with the same release tags produces identical results, regardless of whether the upstream flow has since progressed to newer data.

---

8. Reporting and Communication

8.1 Automated Email Notifications

CBflow provides configurable email notifications triggered by flow events: run start, stage completion, stage failure, and run completion. Email templates are customizable per project and can include embedded QoR summaries, timing slack snapshots, and failure diagnostics.

8.2 AutoPPT – Automated Report Generation

The AutoPPT subsystem generates PowerPoint presentations from flow execution data. These presentations aggregate timing summaries, area reports, power estimates, congestion maps, and signoff status into a standardized format suitable for design review meetings and management reporting.

8.3 Checklist Reports

CBflow maintains per-stage and per-phase quality checklists. The validate subnode of each stage evaluates checklist criteria and records pass/fail results in the stage metrics database. Checklist reports provide a concise summary of design quality at any point in the flow, identifying outstanding issues that require attention before the next phase milestone.

8.4 QoR Metrics Tracking

The RACE engine's stage_metrics table captures quantitative quality-of-results data at every stage: worst negative slack (WNS), total negative slack (TNS), cell count, area, power, wire length, via count, and flow-specific metrics. This data is preserved historically across multiple runs, enabling trend analysis and regression detection.

---

9. Multi-Vendor Support

9.1 Vendor-Agnostic Flow Definition

CBflow separates flow structure from tool implementation. Stage names, dependency relationships, subnode patterns, and status management are identical regardless of which EDA vendor's tools are used for execution. The flow definition is a property of the methodology; the tool binding is a property of the configuration.

9.2 Supported Vendor Matrix

Vendor	Tool	Supported Flows
Synopsys	Fusion Compiler	SYNTH, PNR, SYNTH_PNR, FP, FCFP, ECO
	PrimeTime	STA, POPT
	ICV	PV
	RedHawk	EMIR
	Formality	LEC
	VC-LP	CLP
Cadence	Genus	SYNTH
	Innovus	PNR, FP

Vendor	Tool	Supported Flows
	Tempus	STA
	Voltus	EMIR
	Conformal LP	CLP
Mentor/Siemens	Calibre	PV

9.3 Tool Handler Abstraction

Each vendor/tool/version combination has its own set of TCL handler scripts. Handlers translate abstract stage operations into tool-specific commands, managing tool invocation, option setting, output collection, and error detection for their specific tool.

The handler directory structure enforces version isolation:

```
cmds/<FLOW_TYPE>/<vendor>/<tool>/<version>/
    inputs_subnode_handler.tcl
    init_design_subnode_handler.tcl
    synthesis_subnode_handler.tcl
    place_subnode_handler.tcl
    cts_subnode_handler.tcl
    route_subnode_handler.tcl
    ...
```

9.4 Seamless Tool Switching

Switching from one EDA tool to another requires only a configuration change:

```
tool,vendor    "<vendor_name>"
tool,name      "<tool_name>"
tool,version   "<version>"
```

No flow definition changes, no dependency graph modifications, no GUI reconfiguration. The RACE engine reads the tool binding from the node configuration and routes stage execution to the appropriate handler directory. This enables tool evaluation, migration planning, and mixed-vendor methodologies with minimal operational overhead.

---

10. Technical Specifications

10.1 Technology Stack

Component	Technology
Engine	Python 3.8+ (standard library only)
Handlers	TCL (EDA tool scripting interface)
Configuration	TCL (structured array-based configuration)

Component	Technology
Orchestration	Bash (run creation, environment setup)
Database	SQLite3 (embedded, serverless)
GUI	HTML5, CSS3, JavaScript (zero external dependencies)
Web Server	Python http.server (standard library)

10.2 System Requirements

Requirement	Specification
Operating System	Linux (production), macOS (development)
Python	3.8 or later
TCL	8.5 or later
Disk	Minimal (framework is under 10 MB excluding tool handlers)
Network	Required for GUI access; optional for engine operation
License	No commercial license required for the CBflow framework itself (EDA tool licenses are separate)

10.3 HPC Integration

CBflow integrates with LSF (Load Sharing Facility) for high-performance computing job dispatch. The tool launch configuration maps each flow stage to a resource tier, and each tier defines LSF queue, memory, CPU, and runtime limit parameters. The RACE engine records LSF job IDs in the status database for cross-referencing with cluster management systems.

Four launch modes are supported: - **Local** – Direct subprocess execution on the current host - **Xterm** – Execution in a dedicated terminal window for interactive debugging - **LSF batch** – Submission to an LSF queue for headless execution - **LSF + Xterm** – LSF submission with an attached terminal for monitoring

10.4 Scalability

The RACE engine's SQLite-based state management scales to flows with hundreds of stages and thousands of subnodes. Database operations are transactional and thread-safe. The deterministic UID scheme supports concurrent runs on the same design block without database collision.

The framework supports project-level database consolidation through the structured database path convention, enabling cross-run analytics and project-wide QoR dashboards without additional infrastructure.

---

11. Design Phases and Exit Milestones

11.1 Phase-Driven Methodology

CBflow enforces a structured, phase-driven design methodology that mirrors the natural progression of an ASIC project from early exploration through tapeout. Each phase represents a distinct level of design maturity, with progressively stricter quality requirements and signoff criteria. This phased approach ensures that design teams do not advance to the next stage of implementation until the current phase has met all quality, timing, power, and physical integrity targets.

The framework defines four primary design phases:

Phase P0 – Exploration and Architecture. The initial phase focuses on feasibility analysis, floorplan exploration, and early power/timing estimation. At this stage, the design is fluid – multiple floorplan options may be evaluated, synthesis strategies compared, and clock architectures explored. Quality-of-Results (QoR) targets are preliminary, and the emphasis is on establishing a viable implementation path. CBflow supports P0 by enabling rapid iteration: flows can be retraced from any stage, branches created for what-if analysis, and configurations adjusted without rebuilding the entire workspace.

Phase P1 – Implementation and Optimization. The design transitions from exploration to committed implementation. Floorplan is frozen, synthesis settings are finalized, and placement/routing optimization begins in earnest. Timing closure efforts intensify, and the design team works toward meeting setup and hold constraints across all MMMC scenarios. CBflow's RACE engine tracks per-scenario timing results through the status database, enabling targeted optimization of failing corners without re-running passing scenarios.

Phase P2 – Signoff and Verification. The design enters signoff qualification. All physical verification checks (DRC, LVS, ERC, antenna) must pass. Static timing analysis must close across all corners and modes. IR drop and electromigration must meet specifications. CBflow's parallel verification capabilities – running DRC, LVS, ERC, PERC, and XOR checks simultaneously in the PV flow – significantly reduce signoff turnaround time. The checklist system (Section 12) enforces that all mandatory signoff criteria are satisfied before release.

Phase P3 – Tapeout and Release. The final phase encompasses GDSII generation, final verification, and handoff to the foundry. CBflow's release management system ensures that all mandatory deliverables are present, checksums verified, and release metadata recorded. The release path convention provides a structured archive that enables future reference and ECO implementation.

11.2 Exit Milestones

Each phase transition is governed by exit milestones – formal quality gates that must be satisfied before the design advances. CBflow defines the following exit milestones, each associated with specific stages in the design flow:

FP_EXIT (Floorplan Exit). Validates that the floorplan meets area utilization targets, macro placement is legal, power grid integrity is confirmed, and pin placement satisfies routing requirements. This milestone gates the transition from floorplanning to placement. Required checks include: placement density below target, no overlapping macros, power rail connectivity verified, and I/O pin accessibility confirmed.

PLACE_EXIT (Placement Exit). Confirms that placement optimization has converged. Setup timing targets must be within a defined threshold, congestion hotspots must be below acceptable limits, and cell legalization must be complete. This milestone gates the transition from placement to clock tree synthesis.

CTS_EXIT (Clock Tree Synthesis Exit). Validates clock tree quality including clock skew, insertion delay, clock transition times, and duty cycle distortion. Both setup and hold timing must be within targets after CTS. Power consumption of the clock network is evaluated against the clock power budget.

PRO_EXIT (Post-Route Optimization Exit). The most comprehensive pre-signoff milestone. Validates that post-route timing is closed across all MMMC scenarios, signal integrity (crosstalk) is within limits, and routing congestion is resolved. This milestone gates the transition to signoff verification.

BTO (Block Tapeout). The final technical milestone before release. All physical verification checks must pass (DRC clean, LVS clean, ERC clean). Timing is signed off across all corners. IR drop and electromigration meet specifications. The design is ready for GDSII generation and delivery.

MTO (Mask Tapeout). The organizational milestone confirming that all deliverables have been generated, verified, and archived in the release path. This is the formal handoff point to the foundry or the next level of integration.

11.3 Milestone Enforcement

CBflow enforces exit milestones through its release configuration system. Each milestone defines a set of mandatory checks that must pass before the release_data stage can complete. The release configuration specifies per-phase requirements:

- Mandatory output files that must exist and be non-empty
- Timing report thresholds that must be met
- Physical verification results that must be clean
- QoR metrics that must fall within acceptable ranges

If any mandatory check fails, the release_data stage reports the failure, and the design team must resolve the issue before the milestone can be cleared. This enforcement prevents premature advancement and ensures consistent quality across all design blocks in a project.

---

12. Checklist and Quality Assurance System

12.1 Hierarchical Checklist Framework

CBflow implements a comprehensive checklist system that operates at three levels of granularity, ensuring that quality requirements are captured, tracked, and enforced throughout the design process.

Stage-Level Checklists. Each execution stage in a flow has an associated checklist that defines the expected outcomes and quality criteria for that specific step. For example, the synthesis stage checklist verifies that the gate count is within the area budget, that no unmapped cells remain, and that the timing report shows no critical violations. Stage-level checklists are evaluated automatically when the stage's validate subnode executes.

Phase-Level Checklists. Aggregated checklists that span multiple stages and represent the quality requirements for an entire design phase. A P1 phase checklist, for instance, consolidates timing closure status across all corners, placement quality metrics, and clock tree results into a single quality assessment. Phase-level checklists are evaluated at exit milestones and provide the go/no-go decision criteria for phase transitions.

Project-Level Checklists. Cross-block checklists that ensure consistency across all blocks in a multi-block project. These verify that all blocks use compatible library versions, consistent timing constraints, and aligned power intent specifications. Project-level checklists are managed by the project lead and cannot be modified at the block level.

12.2 Checklist Content

Each checklist item consists of the following attributes:

- **Category** – The functional area (timing, power, physical, verification, methodology)
- **Requirement** – A clear statement of what must be satisfied
- **Check Type** – Automatic (evaluated by the framework) or manual (requires engineer sign-off)
- **Severity** – Critical (blocks release), warning (flagged but non-blocking), or informational
- **Status** – Pass, fail, waived, or not-applicable
- **Evidence** – Reference to the report file, log, or metric that substantiates the result

12.3 Automated Evaluation

CBflow's checklist engine integrates with the RACE status database to automatically evaluate checklist items. When a stage completes, the framework parses the stage's output reports and cross-references them against the checklist criteria. Results are stored in the stage_metrics table of the SQLite database, enabling historical trend analysis across runs.

For automatic checks, the framework supports: - File existence and non-empty validation - Report parsing with configurable regex patterns for metric extraction - Threshold comparison (timing slack, area utilization, power consumption) - Boolean checks (DRC clean, LVS match, no unresolved references)

Manual checks present a sign-off interface where the responsible engineer records their assessment with a timestamp and optional comments. Waiver handling allows checklist items to be waived with documented justification, ensuring traceability without blocking the flow.

12.4 Reporting

Checklist results are available through multiple channels: - The FlowTracer GUI displays checklist status alongside stage properties - The automated email notification system includes checklist summaries in stage completion reports - The AutoPPT system generates checklist result slides for design review presentations - A dedicated checklist report command produces detailed compliance documents suitable for customer deliverables and audit trails

---

13. Cross-Vendor Support Mechanism

13.1 Design Principles

A fundamental architectural decision in CBflow is the complete separation of flow logic from tool-specific implementation. The flow definition – stages, dependencies, subnodes, parallelism – is entirely tool-agnostic. The tool-specific behavior is encapsulated in handler scripts that reside in a well-defined directory hierarchy. This separation enables a project to switch between EDA tool vendors with zero changes to the flow configuration, node dependencies, or execution model.

13.2 Handler Abstraction Layer

Every stage in CBflow executes through a handler script – a TCL file that encapsulates the tool-specific commands, settings, and report parsing for that particular step. Handlers are organized in a hierarchical directory structure:

```
PD/cmds/<FLOW>/<VENDOR>/<TOOL>/<VERSION>/
<stage>_subnode_handler.tcl
```

For example, the placement stage in a PNR flow has separate handlers for Synopsys Fusion Compiler and Cadence Innovus:

```
PD/cmds/PNR/synopsys/fc/v1.0.0/place_subnode_handler.tcl
PD/cmds/PNR/cadence/innovus/v1.0.0/place_subnode_handler.tcl
```

Both handlers implement the same interface – they accept the same arguments (subnode name, run directory, stage name) and produce outputs in the same directory structure. The RACE engine invokes whichever handler the flow configuration specifies, without any awareness of which vendor's tool is being used.

13.3 Tool Configuration Binding

The binding between a flow and its tool backend is controlled exclusively through the flow's node configuration file. Each flow config declares:

- **tool,vendor** – The EDA vendor identifier (synopsys, cadence, mentor)
- **tool,name** – The specific tool within that vendor's suite (fc, innovus, pt, tempus, icv, calibre)
- **tool,version** – The handler version to use (v1.0.0, v1.0.1)
- **supported_tools** – The list of all tool backends available for this flow

The RACE engine reads these declarations from the config at initialization time and resolves the handler path accordingly. There are no hardcoded tool references in the engine, the dashboard, or any framework component. This means that switching from Synopsys to Cadence for a PNR flow requires changing exactly three configuration values – vendor, tool name, and version – with no modifications to the flow graph, status tracking, or GUI.

13.4 Vendor Support Matrix

CBflow currently provides validated handler implementations for the following vendor and tool combinations:

Flow	Synopsys	Cadence	Mentor/ Siemens
SYNTH	Fusion Compiler, Design Compiler	Genus	–
PNR	Fusion Compiler	Innovus	–
SYNTH_PNR	Fusion Compiler	–	–
FP	Fusion Compiler	Innovus	–
FCFP	Fusion Compiler	Innovus	–
STA	PrimeTime	Tempus	–
PV	ICV	–	Calibre
LEC	Formality	–	–
CLP	VC-LP		–

Flow	Synopsys	Cadence	Mentor/ Siemens
		Conformal LP	
ECO	Fusion Compiler, ICC2	–	–
EMIR	RedHawk	Voltus	–
POPT	PrimeTime	–	–

13.5 Handler Interface Contract

All handlers, regardless of vendor, adhere to a strict interface contract that ensures interchangeability:

Input Contract. The handler receives the execution context through a standardized set of arguments and environment variables. The run directory contains all necessary configuration files, input data, and working directories in a tool-agnostic layout. The handler sources the same configuration hierarchy (flow_config, project_config, tech_config, user_config, override_config) regardless of which tool it drives.

Output Contract. Upon completion, the handler produces results in a standardized directory structure. Timing reports, area reports, power reports, and design databases are written to predictable locations that downstream stages can reference without knowledge of which tool generated them. Error messages are written to a standardized log format that the RACE engine can parse for status determination.

Configuration Contract. Tool-specific settings that do not affect the flow structure – such as optimization effort levels, technology-specific switches, or tool-internal parameters – are namespaced under the flow's configuration array. For example, Synopsys-specific compile settings reside under the flow's compile section, while Cadence-specific placement parameters reside under equivalent keys. The handler reads whichever settings are relevant to its tool and ignores the rest.

13.6 Mixed-Vendor Flows

CBflow's architecture supports mixed-vendor flows where different stages use different tools. A common production scenario uses Cadence Genus for synthesis, Synopsys Fusion Compiler for place-and-route, Synopsys PrimeTime for timing signoff, and Mentor Calibre for physical verification. Because each stage independently resolves its handler from the config, different stages in the same project can use different vendors without any special configuration or framework modifications.

This capability is particularly valuable during tool evaluation periods, where a design team may wish to compare results from two vendors on the same stages while keeping the rest of the flow constant. CBflow's branch mechanism enables running the same stage with different tool backends in parallel branches, with results visible side-by-side in the FlowTracer GUI.